

BOOK 15 — JAVA TESTING

JUnit, Mockito & Production-Quality Test Suites (Telugu + English)

Series: Java Interview + Development Mastery Series **Book Number:** 15 of 24 **Versions Covered:** JUnit 5 (Jupiter), Mockito 5.x, Spring Boot Test, Testcontainers **Prerequisites:** Book 04 (exceptions — testing failure paths), Book 07 (functional interfaces — Mockito's fluent API), Book 11 (DI — why constructor injection enables testability), Book 12 (the 3-layer testing preview this book fully expands), Book 13 (JPA — what integration tests exercise) **Next Book:** 16_Microservices.md

How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

Telugu: Book 12, Ch.11 లో మనం Spring Boot testing యొక్క 3 layers ని **preview** చేశాము. ఈ పుస్తకం లో JUnit 5, Mockito, Spring Boot test slices, Testcontainers — అన్నింటినీ లోతుగా నేర్చుకుంటాము. "Code పనిచేస్తుంది" అని believe చేయడం వేరు, **prove** చేయడం వేరు — ఇదే ఈ పుస్తకం యొక్క core theme.

English: Book 12, Ch.11 previewed the 3 layers of Spring Boot testing. This book goes fully deep into JUnit 5, Mockito, Spring Boot test slices, and Testcontainers. Believing your code works is different from proving it does — that distinction is this book's core theme.

Learning Objectives

1. Master JUnit 5's annotations, lifecycle, assertions, and parameterized tests.
2. Master Mockito for mocks, stubs, verification, argument captors, and spies.
3. Write effective, well-structured unit tests using the AAA pattern and FIRST principles.
4. Master Spring Boot's testing slices (`@WebMvcTest` , `@DataJpaTest` , `@SpringBootTest`) in full depth.
5. Use Testcontainers for realistic database integration testing.
6. Apply sound test strategy — knowing what to test at which layer, and what not to over-test.
7. Build a complete, production-quality test suite for a real application.

Table of Contents

Ch.	Title
1	Testing Fundamentals & The Test Pyramid
2	JUnit 5 Deep Dive
3	Mockito Deep Dive
4	Writing Effective Unit Tests
5	Spring Boot Testing Slices

Ch.	Title
6	Testcontainers — Real Database Integration Testing
7	API Testing & Test Strategy
8	Mini Project — Full Test Suite
—	Final Revision Notes, Cheat Sheet, Interview Bank, Revision Plans, Mastery Checklist

CHAPTER 1 — Testing Fundamentals & The Test Pyramid

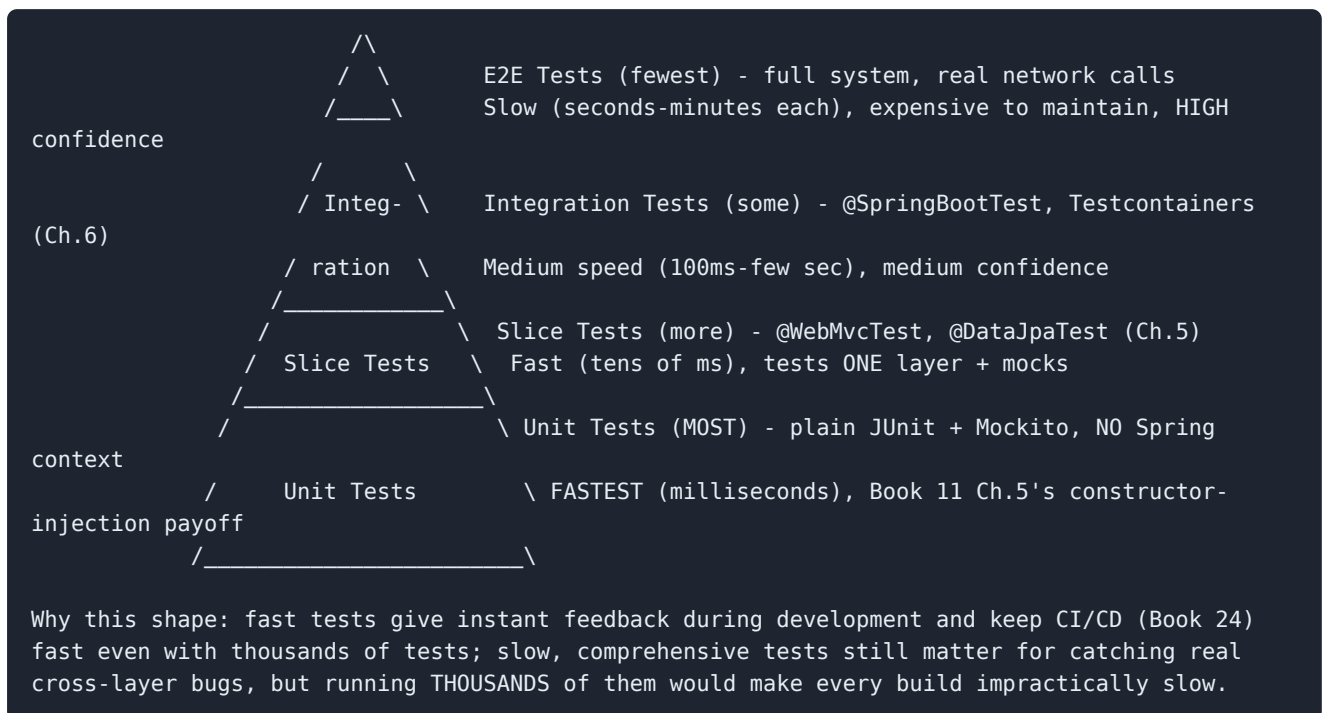
Telugu Explanation

Book 12, Ch.11 లో test pyramid ని (unit → slice → integration, many→few) preview చేశాము. ఈ chapter దాన్ని పూర్తిగా విస్తరిస్తుంది: **Unit test** (ఒక్క class/method, dependencies mock చేయబడతాయి, Spring context లేదు), **Integration test** (multiple components కలిసి, real లేదా realistic dependencies తో), **End-to-End test** (పూర్తి system, real HTTP calls, అత్యంత నెమ్మది కానీ అత్యంత realistic).

Professional English Explanation

Book 12, Ch.11 previewed the test pyramid (unit → slice → integration, many→few). This chapter expands it fully: **Unit test** (one class/method, dependencies mocked, no Spring context), **Integration test** (multiple components together, with real or realistic dependencies), **End-to-End test** (the full system, real HTTP calls, slowest but most realistic).

Diagram — Full Test Pyramid with Speed/Cost Trade-offs



Java Code — The Same Behavior, Tested at 3 Different Levels

```
// Class under test (Book 11's constructor injection payoff, again)
@org.springframework.stereotype.Service
class DiscountService {
    private final CouponRepository couponRepository;
    DiscountService(CouponRepository couponRepository) { this.couponRepository =
couponRepository; }

    double applyDiscount(double amount, String couponCode) {
        return couponRepository.findByCode(couponCode)
            .filter(c -> c.isValid())
            .map(c -> amount * (1 - c.getPercentage()))
            .orElse(amount); // invalid/missing coupon -
no discount
    }
}

// LEVEL 1 - UNIT TEST: fast, isolated, mocked repository
class DiscountServiceUnitTest {
    @org.junit.jupiter.api.Test
    void appliesDiscount_whenCouponValid() {
        CouponRepository mockRepo = org.mockito.Mockito.mock(CouponRepository.class); //
Ch.3
        org.mockito.Mockito.when(mockRepo.findByCode("SAVE10"))
            .thenReturn(java.util.Optional.of(new Coupon("SAVE10", 0.10, true)));

        DiscountService service = new DiscountService(mockRepo);
// plain 'new' - NO Spring context
        double result = service.applyDiscount(100.0, "SAVE10");

        org.junit.jupiter.api.Assertions.assertEquals(90.0, result, 0.001);
// Ch.2's assertions
    }
}

// LEVEL 2 - SLICE TEST: real database (in-memory H2), only the repository layer loaded
@org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest
// Ch.5
class CouponRepositoryDataJpaTest {
    @org.springframework.beans.factory.annotation.Autowired
    private CouponRepository couponRepository;

    @org.junit.jupiter.api.Test
    void findsCouponByCode() {
        couponRepository.save(new Coupon("WELCOME20", 0.20, true));
        java.util.Optional<Coupon> found = couponRepository.findByCode("WELCOME20");
        org.junit.jupiter.api.Assertions.assertTrue(found.isPresent());
    }
}

// LEVEL 3 - INTEGRATION TEST: full application context, real HTTP call, real (test) database
@org.springframework.boot.test.context.SpringBootTest(
    webEnvironment =
org.springframework.boot.test.context.SpringBootTest.WebEnvironment.RANDOM_PORT)
class CheckoutIntegrationTest {
    @org.springframework.beans.factory.annotation.Autowired
    private org.springframework.boot.test.web.client.TestRestTemplate restTemplate;

    @org.junit.jupiter.api.Test
```

```

void checkoutFlow_appliesDiscountCorrectly() {
    // exercises Controller -> Service -> Repository -> real (in-memory or Testcontainers,
    Ch.6) DB, end-to-end
    var response = restTemplate.postForEntity("/api/checkout?coupon=SAVE10", null,
    Double.class);
    org.junit.jupiter.api.Assertions.assertEquals(200, response.getStatusCode().value());
}
}

```

Internal Working

- The **same business behavior** (discount application) is deliberately tested at multiple pyramid levels, but **not redundantly** — the unit test verifies the *discount calculation logic itself* in complete isolation; the slice test verifies the *repository actually finds data correctly* against a real (if in-memory) database; the integration test verifies the *whole request-to-response wiring* is correct — each level catches a **different category** of bug, which is exactly why a healthy test suite has tests at every level, not just one.
- The pyramid's shape (many unit, some slice, few integration/E2E) is directly driven by the speed/realism trade-off: a unit test with zero Spring context (Book 11, Ch.5's testability payoff) runs in single-digit milliseconds; a full `@SpringBootTest` can take seconds to start the application context — multiplied across a codebase with thousands of tests, an inverted pyramid (mostly slow integration tests) would make CI/CD builds (Book 24) impractically slow, directly motivating the shape.
- This chapter's three test classes are **not** three different ways of testing the same thing "just in case" — they're deliberately testing three genuinely different concerns (calculation logic, data access correctness, end-to-end wiring), which is the mature testing mindset this book builds toward, as opposed to a naive "test everything at the highest level for maximum confidence" approach that would be prohibitively slow.

Real-World Example

Telugu: Real production codebases వందల కొద్దీ fast unit tests, కొన్ని dozen slice tests, కేవలం handful of full integration tests కలిగి ఉంటాయి — CI pipeline (Book 24) కొన్ని నిమిషాల్లో complete అవుతుంది, వేల tests ఉన్నా కూడా, ఎందుకంటే most tests fastest level లో ఉంటాయి. English: Real production codebases have hundreds of fast unit tests, a few dozen slice tests, and only a handful of full integration tests — CI pipelines (Book 24) complete in minutes even with thousands of tests, precisely because most tests live at the fastest level.

Interview Answer

"The test pyramid has unit tests (most numerous, fastest, isolated via mocking, enabled by constructor injection, Book 11), slice tests (medium, testing one architectural layer with a real but narrow context), and integration/E2E tests (fewest, slowest, most realistic, full application context). This shape is driven directly by the speed/realism trade-off — thousands of fast unit tests keep CI/CD fast, while a smaller number of slower, more comprehensive tests still catch real cross-layer issues unit tests can't."

Cross Questions

- Q: Why not just write every test as a full `@SpringBootTest` for maximum realism? → A: Speed — full integration tests are dramatically slower to run; at scale (thousands of tests), this would make every CI build impractically slow, which is exactly why the pyramid favors many fast unit tests over many slow integration tests.
- Q: Do the 3 test levels in this chapter's demo test redundant things? → A: No — each verifies a genuinely different concern: calculation logic (unit), data access correctness (slice), end-to-end request wiring (integration) — a bug in any one layer might only be caught by the test level actually exercising that layer.
- Q: What specifically enables fast, Spring-context-free unit testing? → A: Constructor injection (Book 11, Ch.5) — a class's dependencies are ordinary constructor parameters, so a test can supply mocks directly via `new`, with no Spring container involved at all.

Tricky Questions

- Q: If a codebase has 90% of its tests as slow `@SpringBootTest` integration tests and only 10% unit tests, is that automatically "bad"? → A: Not automatically wrong in every case, but it's a strong signal worth investigating — it likely indicates either heavy reliance on field injection (Book 11, Ch.5, hampering easy unit testing) or a team default of "test everything at the highest level," both of which cost real CI/CD speed and slow feedback loops at scale.
- Q: Can a bug exist that NO level of the pyramid would catch? → A: Yes, in principle — genuinely untested code paths, or bugs only manifesting under production-scale load/data/concurrency (Book 08) that no reasonably-sized test suite reproduces; this is exactly why production monitoring/observability (Book 12, Ch.10) complements, rather than replaces, testing.

Coding Exercise

L1: Write the same feature's test at all 3 pyramid levels, as shown in this chapter's demo, for a domain of your choice. **L2:** Time each test level's execution and compare, illustrating the speed trade-off concretely. **L3:** Identify, in an existing codebase (or a provided sample), tests that are redundant across levels versus tests that each catch something genuinely different. **L4 (Interview):** Explain the test pyramid and the speed/realism trade-off driving its shape. **L5 (Senior):** Review a codebase with an inverted pyramid (mostly slow integration tests) and propose a rebalancing plan. **L6 (Mastery):** Explain, from memory, why constructor injection (Book 11) is what makes the base of the pyramid possible at all.

CHAPTER 2 — JUnit 5 Deep Dive

Telugu Explanation

JUnit 5 (Jupiter) modern Java testing యొక్క foundation. Lifecycle annotations:

`@BeforeEach` / `@AfterEach` (ప్రతి test method కి ముందు/తర్వాత), `@BeforeAll` / `@AfterAll` (class కి ఒక్కసారి, `static` కావాలి). Assertions: `assertEquals`, `assertTrue`, `assertThrows` (Book 04's exceptions test చేయడానికి), `assertAll` (multiple assertions, అన్నీ run అయ్యి, అన్ని failures ఒకేసారి report అవ్వడానికి).

Professional English Explanation

JUnit 5 (Jupiter) is the foundation of modern Java testing. Lifecycle annotations:

`@BeforeEach` / `@AfterEach` (run before/after each test method), `@BeforeAll` / `@AfterAll` (run once per class, must be `static`). Assertions: `assertEquals`, `assertTrue`, `assertThrows` (for testing Book 04's exceptions), `assertAll` (grouping multiple assertions so all run and all failures are reported together, not just the first).

Java Code

```
import org.junit.jupiter.api.*;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.*;
import static org.junit.jupiter.api.Assertions.*;

class JUnit5DeepDiveDemo {

    static java.util.List<String> executionLog = new java.util.ArrayList<>();

    @BeforeAll
    static void setUpClass() { // ONCE, before ANY test in
this class - must be static
        executionLog.add("beforeAll");
        System.out.println("Setting up shared, expensive resources (once) - e.g., a
Testcontainers DB, Ch.6");
    }

    @BeforeEach
    void setUp() { // before EVERY test
method - fresh state each time
        executionLog.add("beforeEach");
    }

    @Test
    @DisplayName("Addition of two positive numbers") // human-readable test
    report name
    void testAddition() {
        assertEquals(5, 2 + 3, "2 + 3 should equal 5"); // message shown
ONLY if assertion fails
    }

    @Test
    void testExceptionThrown() {
        BankAccount account = new BankAccount(100.0);
        InsufficientFundsException ex = assertThrows( // Book 04's
exception testing
            InsufficientFundsException.class,
            () -> account.withdraw(500.0));
        assertEquals("Insufficient funds", ex.getMessage()); // can inspect
the caught exception further
    }

    @Test
    void testMultipleAssertionsTogether() {
        BankAccount account = new BankAccount(1000.0);
        assertAll("account properties", // ALL run,
ALL failures reported together
            () -> assertEquals(1000.0, account.getBalance()),
            () -> assertTrue(account.isActive()),
            () -> assertNotNull(account.getId())
        );
        // If assertEquals fails here, assertTrue and assertNotNull STILL run - unlike
sequential asserts
        // where the first failure would stop the test immediately, hiding info about the OTHER
checks
    }

    @ParameterizedTest // ONE
```



```

test method, MANY input sets
@ValueSource(doubles = {10.0, 50.0, 99.99})
void testValidWithdrawal(double amount) {
    BankAccount account = new BankAccount(100.0);
    assertDoesNotThrow(() -> account.withdraw(amount));
}

@ParameterizedTest
@CsvSource({"100.0, 50.0, 50.0", "200.0, 200.0, 0.0", "500.0, 100.0, 400.0"}) //
balance, withdraw, expected remaining
void testWithdrawalCalculatesCorrectRemainingBalance(double balance, double withdraw,
double expected) {
    BankAccount account = new BankAccount(balance);
    account.withdraw(withdraw);
    assertEquals(expected, account.getBalance(), 0.001);
}

@AfterEach
void tearDown() { executionLog.add("afterEach"); }

@AfterAll
static void tearDownClass() {
    executionLog.add("afterAll");
    System.out.println("Tearing down shared resources (once)");
}
}

class InsufficientFundsException extends RuntimeException { InsufficientFundsException(String
m) { super(m); } }
class BankAccount {
    private double balance; private boolean active = true; private final String id =
java.util.UUID.randomUUID().toString();
    BankAccount(double balance) { this.balance = balance; }
    void withdraw(double amount) {
        if (amount > balance) throw new InsufficientFundsException("Insufficient funds");
        balance -= amount;
    }
    double getBalance() { return balance; }
    boolean isActive() { return active; }
    String getId() { return id; }
}
}

```

Internal Working

- `@BeforeAll` / `@AfterAll` must be **static** because they run **once for the entire test class**, before any test instance necessarily exists yet (JUnit 5 by default creates a **new test class instance for every single test method**, precisely to guarantee test isolation — one test's mutated instance state can never leak into another test) — this is exactly why `@BeforeEach` (instance method, runs per-test on a fresh instance) is the right place for per-test setup, while `@BeforeAll` (static, runs once total) suits genuinely expensive, safely-shared setup like starting a Testcontainers database (Ch.6).
- `assertAll()` groups assertions using a fundamentally different failure-reporting strategy than writing them sequentially: sequential assertions **stop at the first failure** (subsequent lines never execute), potentially hiding that *multiple* things are actually wrong; `assertAll()` executes **every** supplied assertion regardless of earlier failures and reports **all** failures together in one

combined report — genuinely more useful for diagnosing multi-property object state in one test run instead of a slow "fix one, rerun, discover the next failure" cycle.

- `@ParameterizedTest` runs the **same test logic** against multiple input sets, reported as **separate individual test results** (not one pass/fail for the whole set) — this is both DRY (Book 02's principle applied to test code itself, avoiding near-duplicate test methods differing only in input values) and more informative than a hand-rolled loop inside one `@Test` method (which would just report one aggregate pass/fail, obscuring which specific input actually failed).

Real-World Example

Telugu: Real production test suites `@ParameterizedTest` వాడి, ఒకే validation logic ని dozens of edge-case inputs మీద test చేస్తారు (empty string, null, boundary values, Unicode) — ఒక్కో దానికీ వేరే `@Test` method రాయడం కంటే చాలా maintainable. English: Real production test suites use `@ParameterizedTest` to test the same validation logic against dozens of edge-case inputs (empty string, null, boundary values, Unicode) — far more maintainable than writing a separate near-duplicate `@Test` method for each one.

Interview Answer

"JUnit 5's lifecycle annotations are `@BeforeEach` / `@AfterEach` (per test method, on a fresh instance, since JUnit creates a new test class instance per test for isolation) and `@BeforeAll` / `@AfterAll` (once per class, must be static, for genuinely expensive shared setup). `assertThrows` tests exception-throwing behavior directly (Book 04), `assertAll` groups assertions so all run and all failures report together instead of stopping at the first. `@ParameterizedTest` runs the same test logic across multiple input sets, each reported as an individual result, avoiding near-duplicate test methods."

Cross Questions

- Q: Why must `@BeforeAll`/`@AfterAll` methods be static? → A: They run once for the entire test class, potentially before any test instance exists (JUnit 5 creates a fresh instance per test method by default), so they can't rely on instance state.
- Q: Why does JUnit 5 create a new test class instance for every test method? → A: To guarantee test isolation — one test's mutated instance state can never accidentally affect another test's execution.
- Q: What's the practical benefit of `assertAll()` over writing sequential assertions? → A: Sequential assertions stop at the first failure, hiding whether other things are also wrong; `assertAll()` runs every assertion and reports all failures together in one pass.

Tricky Questions

- Q: If a `@BeforeEach` method throws an exception, does the corresponding `@Test` method still run? → A: No — the test is reported as failed/errored, and its body never executes, since setup failed before the test itself could meaningfully begin.
- Q: Can `@ParameterizedTest` sources be more complex than simple literal values, like objects or method-provided data? → A: Yes — `@MethodSource` can reference a static method returning a stream of arguments (including complex objects), and `@CsvFileSource` can read parameters

from an external CSV file, both useful for more complex or externally-maintained test data sets beyond simple inline values.

Coding Exercise

L1: Write a test class using all 4 lifecycle annotations, printing execution order to confirm your understanding. **L2:** Use `assertThrows` to test a custom exception, including inspecting its message. **L3:** Convert 3 near-duplicate `@Test` methods into one `@ParameterizedTest` with `@CsvSource`. **L4 (Interview):** Explain JUnit 5's lifecycle annotations and why `@BeforeAll` must be static. **L5 (Senior):** Review a test class with 15 near-identical test methods differing only in input values — refactor it using `@ParameterizedTest`. **L6 (Mastery):** Explain, from memory, why JUnit 5 creates a new test instance per test method, and how that connects to test isolation.

CHAPTER 3 — Mockito Deep Dive

Telugu Explanation

Mockito real dependencies బదులు **fake objects (mocks)** create చేసే library — unit test ని దాని dependencies నుండి పూర్తిగా isolate చేయడానికి. `mock()` (fake object create చేయడం), `when().thenReturn()` (stubbing — mock ఏమి చేయాలో చెప్పడం), `verify()` (mock చేసిన ఏ method ఎన్నిసార్లు call అయ్యిందో check చేయడం), `ArgumentCaptor` (mock కి పంపిన actual arguments ని capture చేసి inspect చేయడం).

Professional English Explanation

Mockito creates **fake objects (mocks)** in place of real dependencies — fully isolating a unit test from its dependencies. `mock()` (create a fake object), `when().thenReturn()` (stubbing — telling the mock what to do), `verify()` (checking which methods were called on the mock, how many times), `ArgumentCaptor` (capturing and inspecting the actual arguments passed to a mock).

Java Code

```
import org.mockito.*;
import org.junit.jupiter.api.*;
import java.util.*;
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;

class MockitoDeepDiveDemo {

    interface EmailService { void send(String to, String subject, String body); }
    interface OrderRepository {
        Optional<Order> findById(Long id);
        Order save(Order order);
    }
    record Order(Long id, String customerEmail, String status) {}

    static class OrderService {
        private final OrderRepository repository;
        private final EmailService emailService;
        OrderService(OrderRepository repository, EmailService emailService) {
            this.repository = repository; this.emailService = emailService;
        }
        void shipOrder(Long orderId) {
            Order order = repository.findById(orderId).orElseThrow();
            Order shipped = new Order(order.id(), order.customerEmail(), "SHIPPED");
            repository.save(shipped);
            emailService.send(order.customerEmail(), "Your order shipped!", "Order #" + orderId
+ " is on its way");
        }
    }

    @Test
    void shipOrder_updatesStatusAndSendsEmail() {
        // ARRANGE: create mocks, stub their behavior
        OrderRepository mockRepo = mock(OrderRepository.class);
        EmailService mockEmail = mock(EmailService.class);
        Order existingOrder = new Order(1L, "ravi@example.com", "PENDING");
        when(mockRepo.findById(1L)).thenReturn(Optional.of(existingOrder)); //
STUBBING

        OrderService service = new OrderService(mockRepo, mockEmail);

        // ACT
        service.shipOrder(1L);

        // ASSERT: verify the mock was called correctly (behavior verification, not just
return-value checking)
        ArgumentCaptor<Order> orderCaptor = ArgumentCaptor.forClass(Order.class); //
captures the ACTUAL argument passed
        verify(mockRepo).save(orderCaptor.capture());
        assertEquals("SHIPPED", orderCaptor.getValue().status());
        // inspect what was ACTUALLY saved

        verify(mockEmail, times(1)).send(eq("ravi@example.com"), anyString(), anyString());
        // exactly once, with matchers
        verify(mockRepo, never()).save(argThat(o -> o.status().equals("CANCELLED")));
        // negative verification
    }
}
```

```

@Test
void shipOrder_throwsWhenOrderNotFound() {
    OrderRepository mockRepo = mock(OrderRepository.class);
    EmailService mockEmail = mock(EmailService.class);
    when(mockRepo.findById(999L)).thenReturn(Optional.empty());
// stub: not found

    OrderService service = new OrderService(mockRepo, mockEmail);

    assertThrows(java.util.NoSuchElementException.class, () -> service.shipOrder(999L));
    verifyNoInteractions(mockEmail);
// email must NEVER be sent
}

// @Spy: wraps a REAL object, but lets you selectively override/verify specific method
calls
@Test
void spyDemo() {
    List<String> realList = new ArrayList<>();
    List<String> spyList = spy(realList);

    spyList.add("real item"); // actually executes on
the REAL list
    assertEquals(1, spyList.size()); // real behavior -
genuinely has 1 item

    doReturn(100).when(spyList).size(); // OVERRIDE just
this one method's behavior
    assertEquals(100, spyList.size()); // now returns the
STUBBED value instead
}
}

```

Internal Working

- `mock()` creates a **dynamically-generated proxy object** (conceptually similar to Book 11, Ch.8's AOP/Spring Data JPA proxies) that implements the target interface/class but, by default, does **nothing** — every method returns a sensible default (null, 0, false, empty Optional/Collection) unless explicitly **stubbed** with `when(...).thenReturn(...)`; this is precisely why forgetting to stub a mock's method and then calling it produces a silent `null` /default rather than an error — a common source of confusing test failures for developers new to mocking.
- `verify()` implements **behavior verification** — a fundamentally different testing style from **state verification** (checking a return value's correctness): rather than asking "did this method return the right value," it asks "was this dependency method actually **called**, how many times, with what arguments" — essential for testing methods with **side effects but no meaningful return value** (like `shipOrder()` above, which returns `void` but has the crucial side effect of calling `save()` and `send()`).
- `spy()` differs fundamentally from `mock()`: a mock starts with **no real behavior at all** (everything must be stubbed); a spy wraps a **genuinely real object**, executing real method calls by default, letting you selectively override just specific methods via `doReturn().when()` — useful when you want mostly-real behavior with one or two specific interactions overridden, though spies are used more sparingly than plain mocks in idiomatic unit testing, since relying on real behavior can reintroduce some of the coupling/slowness mocking was meant to eliminate.

Real-World Example

Telugu: `shipOrder()` వంటి void-returning, side-effect-heavy methods (Book 12's Service layer లో extremely common) — వాటిని test చేయడానికి `verify()` -based behavior verification తప్పనిసరి, ఎందుకంటే return value లేదు, side effects మాత్రమే observable. English: void -returning, side-effect-heavy methods (extremely common in Book 12's Service layer) genuinely require `verify()` -based behavior verification to test at all — there's no meaningful return value to check, only side effects to observe and confirm.

Interview Answer

"Mockito creates dynamic proxy mocks that do nothing by default until explicitly stubbed via `when().thenReturn()`. `verify()` performs behavior verification — checking a mock method was actually called, how many times, with what arguments — essential for testing void/side-effect-heavy methods where there's no return value to check via state-based assertions. `ArgumentCaptor` captures the actual arguments passed to a mock for detailed inspection. `spy()` wraps a genuinely real object, executing real behavior by default while letting specific methods be selectively overridden — a different, more selective tool than a plain mock."

Cross Questions

- Q: What does an unstubbed mock method return when called? → A: A sensible default based on the return type — null for objects, 0 for numbers, false for booleans, empty Optional/Collection — never an error, which is why forgetting to stub something needed can produce a confusing silent failure.
- Q: Why is behavior verification (`verify()`) necessary for some tests, where simple return-value assertions aren't enough? → A: For void-returning or side-effect-only methods, there's no return value to check via state assertions — you can only confirm correctness by verifying the expected interactions with dependencies actually occurred.
- Q: How does `spy()` differ from `mock()`? → A: A mock has no real behavior until stubbed; a spy wraps a real object and executes real behavior by default, with specific methods selectively overridable — a mock is "nothing until told," a spy is "everything real until told otherwise."

Tricky Questions

- Q: If you call `when(mockRepo.findById(1L)).thenReturn(...)` but the code under test actually calls `findById(2L)`, what happens? → A: The mock returns the default (an empty Optional, or null depending on the method's declared return type) for the unstubbed argument `2L` — Mockito stubs are argument-specific by default, a genuine, common source of confusing "why didn't my stub work" test failures when the wrong argument value was used in the stub setup.
- Q: Is it good practice to use `spy()` as your default choice instead of `mock()` for most unit tests? → A: No — plain `mock()` is the idiomatic default; `spy()` should be reserved for specific cases needing mostly-real behavior with a few overridden interactions, since over-relying on spies can reintroduce real dependency behavior/coupling that unit testing is meant to eliminate in the first place.

Coding Exercise

L1: Write a Mockito-based unit test for a void-returning service method, using `verify()` to confirm the expected side effects. **L2:** Use `ArgumentCaptor` to capture and assert on the actual object passed to a mocked repository's `save()` method. **L3:** Reproduce the "unstubbed argument returns default" gotcha by stubbing for one argument value and calling with a different one. **L4 (Interview):** Explain the difference between state verification and behavior verification, and when each is appropriate. **L5 (Senior):** Review a test suite overusing `spy()` where plain `mock()` would be more appropriate — explain the risk and refactor. **L6 (Mastery):** Explain, from memory, exactly what an unstubbed mock method returns and why that behavior exists.

CHAPTER 4 — Writing Effective Unit Tests

Telugu Explanation

Good unit tests **AAA pattern** follow: **Arrange** (setup — mocks, test data), **Act** (the actual method call being tested), **Assert** (verify the outcome). **FIRST principles**: **F**ast, **I**ndependent (ఒక test మరో test మీద ఆధారపడకూడదు), **R**epeatable (ఎప్పుడు run చేసినా అదే result), **S**elf-validating (pass/fail clear గా, manual inspection అవసరం లేకుండా), **T**imely (code తోపాటే రాయాలి, తర్వాత కాదు).

Professional English Explanation

Good unit tests follow the **AAA pattern**: **Arrange** (setup — mocks, test data), **Act** (the actual method call under test), **Assert** (verify the outcome). **FIRST principles**: **F**ast, **I**ndependent (no test depends on another's execution/state), **R**epeatable (same result every run, in any environment), **S**elf-validating (clear pass/fail, no manual inspection needed), **T**imely (written alongside the code, not long after).

Java Code — Good vs Bad Test Examples

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class EffectiveTestsDemo {

    // BAD: unclear structure, testing multiple unrelated things, poor naming
    @Test
    void test1() {
        BankAccount acc = new BankAccount(100.0);
        acc.withdraw(30.0);
        assertEquals(70.0, acc.getBalance());
        acc.withdraw(1000.0); // this throws - but the test
method NAME gives no hint why
    }

    // GOOD: clear AAA structure, ONE behavior per test, descriptive name
    @Test
    void withdraw_reducesBalance_whenSufficientFunds() {
        // Arrange
        BankAccount account = new BankAccount(100.0);
        // Act
        account.withdraw(30.0);
        // Assert
        assertEquals(70.0, account.getBalance(), 0.001);
    }

    @Test
    void withdraw_throwsInsufficientFundsException_whenAmountExceedsBalance() {
        // Arrange
        BankAccount account = new BankAccount(100.0);
        // Act & Assert (combined here since assertThrows needs the action)
        assertThrows(InsufficientFundsException.class, () -> account.withdraw(1000.0));
    }

    // INDEPENDENCE violation (BAD): relies on shared, mutable static state across tests
    static BankAccount sharedAccount = new BankAccount(500.0); // DANGEROUS - shared
    across tests!

    @Test
    void badTest_dependsOnExecutionOrder_1() {
        sharedAccount.withdraw(100.0);
        assertEquals(400.0, sharedAccount.getBalance()); // passes ONLY if
this runs first
    }

    @Test
    void badTest_dependsOnExecutionOrder_2() {
        assertEquals(500.0, sharedAccount.getBalance()); // FAILS if the
test above already ran - order-dependent!
    }

    // GOOD: fresh, independent state per test (via @BeforeEach or local setup)
    private BankAccount freshAccount;

    @BeforeEach
    void setUp() { freshAccount = new BankAccount(500.0); } // NEW instance for
EVERY test - true independence
```

```

@Test
void goodTest_independent_1() {
    freshAccount.withdraw(100.0);
    assertEquals(400.0, freshAccount.getBalance());
}

@Test
void goodTest_independent_2() {
    assertEquals(500.0, freshAccount.getBalance());
    // ALWAYS passes,
    regardless of execution order
}
}

```

Internal Working

- The **AAA pattern's** value isn't merely cosmetic — a test that clearly separates Arrange/Act/Assert is dramatically easier to debug when it fails (you immediately know whether the *setup* was wrong, the *action* behaved unexpectedly, or the *expectation* itself was incorrect) compared to a tangled test mixing all three concerns together, exactly like Book 02's Single Responsibility Principle applied to test code — one test method should verify **one specific behavior**, not several unrelated ones bundled together.
- **Test independence (the "I" in FIRST)** is genuinely critical, not just a nice-to-have: shared mutable state (like `sharedAccount` in the bad example) makes tests **order-dependent** — they pass or fail based on which other tests happened to run first, a genuinely serious problem since most test runners don't guarantee execution order, and even when they do, relying on it makes the test suite fragile to reordering, parallelization (a real, common CI optimization), or someone adding a new test in between.
- **Repeatability** requires eliminating hidden dependencies on the external environment — a test relying on the current date/time (without controlling it explicitly), random values (without a fixed seed), or network calls to a real, potentially-flaky external service all violate repeatability, and are exactly the kinds of "flaky test" root causes that erode a team's trust in their test suite over time (a real, serious organizational cost when tests are ignored because "they fail randomly anyway").

Real-World Example

Telugu: Real production test suites లో "flaky tests" (కొన్నిసార్లు pass, కొన్నిసార్లు fail, code మారకుండానే) — దాదాపు ఎప్పుడూ FIRST principles violate చేయడం వల్లనే వస్తాయి (shared state, timing dependencies, external calls) — ఇవి team యొక్క test suite మీద trust ని నాశనం చేస్తాయి. English: "Flaky tests" (sometimes pass, sometimes fail, with no code change) in real production test suites almost always trace back to violating FIRST principles — shared state, timing dependencies, external calls — and they genuinely erode a team's trust in their entire test suite over time, a real organizational cost worth actively preventing.

Interview Answer

"Effective unit tests follow AAA structure (Arrange, Act, Assert) — clearly separated for easy debugging — and FIRST principles: Fast, Independent (no shared mutable state between tests), Repeatable (no hidden dependencies on time/randomness/network), Self-validating (clear automated pass/fail), Timely (written alongside the code). Test independence is especially critical —

shared mutable state between tests creates order-dependent, fragile tests that can pass or fail based on execution order or parallelization, a common real source of eroded trust in a test suite."

Cross Questions

- Q: Why is the AAA pattern valuable beyond just readability? → A: When a test fails, clear AAA structure immediately tells you whether the failure is in setup, the action itself, or the expectation — dramatically speeding up debugging compared to tangled test logic.
- Q: What's the practical risk of tests sharing mutable static state? → A: Tests become order-dependent — passing or failing based on which other tests ran first, fragile to reordering, parallelization, or new tests being added, a genuine source of flaky, hard-to-trust test suites.
- Q: What are common causes of "flaky" (intermittently failing) tests? → A: Shared mutable state between tests, hidden dependencies on current time/date without controlling it, unseeded randomness, and calls to real (potentially unreliable) external services — all violations of the FIRST principles.

Tricky Questions

- Q: If a test passes reliably when run alone but fails when run as part of the full suite, what's the most likely root cause? → A: A test independence violation — shared mutable state (static fields, a shared database row, a shared file) that another test in the suite modifies, causing order-dependent behavior invisible when running the test in isolation.
- Q: Is testing multiple related assertions in one test method always a violation of "one behavior per test"? → A: Not necessarily — `assertAll()` (Ch.2) grouping multiple assertions about the *same* logical outcome (e.g., several properties of one resulting object) is fine and often clearer than multiple near-identical test methods; the principle is about not testing *unrelated* behaviors in one test, not an absolute one-assertion-per-test rule.

Coding Exercise

L1: Rewrite a tangled, poorly-structured test using clear AAA structure and a descriptive name. **L2:** Reproduce the shared-static-state test-independence violation, observing order-dependent failures, then fix it with `@BeforeEach`. **L3:** Identify and fix a test relying on an uncontrolled `LocalDate.now()` (Book 07, Ch.10) call, making it deterministic instead. **L4 (Interview):** Explain the AAA pattern and all 5 FIRST principles. **L5 (Senior):** Diagnose a reported "flaky test" in a CI pipeline (fails intermittently, no code changes) — walk through your investigation process using the FIRST principles as a checklist. **L6 (Mastery):** Explain, from memory, why test independence specifically matters for parallelized CI test execution.

CHAPTER 5 — Spring Boot Testing Slices

Telugu Explanation

Book 12, Ch.11 లో `@WebMvcTest` / `@SpringBootTest` preview చేశాము. ఇప్పుడు slice testing యొక్క పూర్తి family చూద్దాం: `@WebMvcTest` (controller layer మాత్రమే), `@DataJpaTest` (repository layer మాత్రమే, in-memory DB తో, transaction ప్రతి test తర్వాత automatic గా rollback అవుతుంది), `@JsonTest` (Jackson serialization మాత్రమే), ఒక్కొక్కటి **narrow, fast context** load చేస్తుంది, పూర్తి application కాదు.

Professional English Explanation

Book 12, Ch.11 previewed `@WebMvcTest` / `@SpringBootTest` . Now, the full slice-testing family: `@WebMvcTest` (web/controller layer only), `@DataJpaTest` (repository layer only, in-memory DB, each test's transaction automatically rolled back afterward), `@JsonTest` (Jackson serialization only) — each loading a **narrow, fast** context, not the full application.

Java Code

```
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.test.autoconfigure.json.JsonTest;
import org.springframework.boot.test.json.JacksonTester;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.context.jdbc.Sql;
import org.junit.jupiter.api.Test;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;
import static org.assertj.core.api.Assertions.assertThat;

// @WebMvcTest: web layer ONLY - controller, filters, exception handlers - service is MOCKED
@WebMvcTest(TaskController.class)
class TaskControllerSliceTest {
    @Autowired private MockMvc mockMvc;
    @MockBean private TaskService taskService; // fake - controller
    tested in isolation from real logic

    @Test
    void getTask_returns404_whenNotFound() throws Exception {
        org.mockito.Mockito.when(taskService.findById(999L)).thenReturn(new
TaskNotFoundException("999"));

        mockMvc.perform(get("/api/tasks/999"))
            .andExpect(status().isNotFound()); // Book 10,
Ch.1's status code, via Ch.6's global handler
    }

    @Test
    void createTask_returns400_whenTitleBlank() throws Exception {
        mockMvc.perform(post("/api/tasks")
            .contentType("application/json")
            .content("{\"title\": \"\"}")) // triggers
Book 12, Ch.5's @Valid
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.message").value("Validation failed"));
    }
}

// @DataJpaTest: repository layer ONLY, real in-memory DB, each test AUTO-ROLLED-BACK
@DataJpaTest
class TaskRepositorySliceTest {
    @Autowired private TaskRepository taskRepository;

    @Test
    void findsIncompleteTasks() {
        taskRepository.save(new Task("Buy groceries", false));
        taskRepository.save(new Task("Finish report", true));

        java.util.List<Task> incomplete = taskRepository.findByCompletedFalse(); // Book
13's derived query

        assertThat(incomplete).hasSize(1);
        assertThat(incomplete.get(0).getTitle()).isEqualTo("Buy groceries");
        // NO manual cleanup needed - @DataJpaTest wraps each test in a transaction, ROLLED
BACK automatically after
    }
}
```

```

    }

    @Test
    @Sql("/test-data.sql") // load
    specific test data before this ONE test
    void findsPreloadedTestData() {
        assertThat(taskRepository.findAll()).isNotEmpty();
    }
}

// @JsonTest: ONLY Jackson serialization/deserialization, no web, no database at all
@JsonTest
class TaskDtoToJsonTest {
    @Autowired private JacksonTester<TaskResponse> json; //
    Book 12, Ch.4's DTO

    @Test
    void serializesCorrectly() throws Exception {
        TaskResponse response = new TaskResponse(1L, "Buy groceries", false);
        assertThat(json.write(response)).hasJsonPathStringValue("$.title");

        assertThat(json.write(response)).extractingJsonPathStringValue("$.title").isEqualTo("Buy
        groceries");
    }
}

```

Internal Working

- Each `@...Test` slice annotation configures Spring Boot to auto-configure **only** the beans relevant to that specific concern — `@WebMvcTest` loads controllers, `@ControllerAdvice` (Book 12, Ch.6), and MVC infrastructure, but explicitly does **not** load `@Service` / `@Repository` beans (hence `@MockBean` being required for the service dependency) — this selective loading is precisely what makes slice tests dramatically faster than a full `@SpringBootTest` while still testing real Spring MVC request-handling machinery (unlike a plain unit test, which wouldn't exercise routing, JSON binding, or `@Valid` at all).
- `@DataJpaTest` 's **automatic transaction rollback** is a genuinely important, easy-to-overlook detail: each test method runs inside its own transaction (Book 13, Ch.10's `@Transactional` mechanics) that is **rolled back** after the test completes — meaning tests never need manual cleanup code, and data from one test never leaks into another (directly satisfying Ch.4's test-independence principle), even though tests genuinely hit a real (in-memory, by default) database.
- `MockMvc` performs simulated HTTP requests **without starting a real embedded server** — it directly invokes Spring MVC's request-processing machinery in-process, which is both why it's fast (no real network I/O) and why it can still meaningfully test real routing, serialization, validation, and exception-handling behavior, distinguishing it clearly from `TestRestTemplate` (Book 12, Ch.11), which does make genuine HTTP calls against a real running server (used in full `@SpringBootTest` integration tests).

Real-World Example

Telugu: Real Spring Boot projects `@WebMvcTest` వాడి, controller layer యొక్క routing/validation/exception-handling correctness test చేస్తారు, service logic నుండి పూర్తిగా isolate చేసి —

ಇದಿ Book 12 ಅಂತ್ಯದಲ್ಲಿ ಮಿರು build చేಸిన REST API code ನಿ fast ನ, comprehensive ನ test చేయడానికి standard approach. English: Real Spring Boot projects use `@WebMvcTest` to test controller-layer routing/validation/exception-handling correctness, fully isolated from service logic — the standard approach for fast, comprehensive testing of exactly the REST API code built throughout Book 12.

Interview Answer

"Spring Boot's testing slices — `@WebMvcTest`, `@DataJpaTest`, `@JsonTest` — each load only the narrow set of beans relevant to that specific concern, dramatically faster than a full `@SpringBootTest` while still exercising real Spring infrastructure (unlike plain unit tests). `@WebMvcTest` loads controllers/exception handlers with services mocked via `@MockBean`, testing via `MockMvc` (simulated requests, no real server). `@DataJpaTest` loads only the repository layer against a real (typically in-memory) database, with each test's transaction automatically rolled back afterward, ensuring test independence with zero manual cleanup."

Cross Questions

- Q: Why does `@WebMvcTest` require `@MockBean` for the service layer? → A: Because `@WebMvcTest` deliberately doesn't load `@Service` beans at all — only web-layer infrastructure — so the controller's service dependency must be supplied as a mock for the context to wire correctly.
- Q: What guarantees test independence in `@DataJpaTest` without manual cleanup code? → A: Each test runs inside its own transaction, automatically rolled back after the test completes — data never persists or leaks between tests.
- Q: What's the key difference between `MockMvc` and `TestRestTemplate`? → A: `MockMvc` simulates requests in-process without a real server (fast, used in slice tests); `TestRestTemplate` makes genuine HTTP calls against a real running server (used in full `@SpringBootTest` integration tests, Book 12 Ch.11).

Tricky Questions

- Q: If a `@DataJpaTest` test explicitly calls something that commits the transaction mid-test (a rare but possible scenario), does automatic rollback still fully undo everything? → A: Not necessarily — if a transaction is explicitly committed within the test, that data genuinely persists beyond the automatic rollback boundary; this is an edge case worth being aware of, since it can silently violate the usual "no cleanup needed" assumption.
- Q: Can `@WebMvcTest` catch a bug in the actual business logic inside a mocked `@Service` method? → A: No — since the service is entirely mocked (stubbed to return whatever the test specifies), `@WebMvcTest` can only catch bugs in the web/controller layer itself (routing, validation, serialization, exception translation), never in the real service logic, which is exactly why unit tests (Ch.1's Level 1) are still needed separately for that.

Coding Exercise

L1: Write a `@WebMvcTest` for a controller, mocking its service dependency and verifying both a success and an error response. **L2:** Write a `@DataJpaTest` verifying a derived query method,

confirming no manual cleanup is needed across multiple test methods. **L3:** Write a `@JsonTest` verifying a DTO serializes with the expected field names and values. **L4 (Interview):** Explain what each of `@WebMvcTest`, `@DataJpaTest`, and `@JsonTest` loads and doesn't load. **L5 (Senior):** Design the full slice-test coverage plan for a new feature spanning controller, service, and repository layers, specifying which test type covers which layer. **L6 (Mastery):** Explain, from memory, why `@DataJpaTest`'s automatic rollback is critical for test independence (Ch.4), and the edge case where it might not fully apply.

CHAPTER 6 — Testcontainers: Real Database Integration Testing

Telugu Explanation

@DataJpaTest default గా H2 (in-memory DB) వాడుతుంది — fast, కానీ production DB (PostgreSQL, MySQL) తో **100% identical గా behave అవుదు** (SQL dialect differences, constraint behaviors).

Testcontainers ఒక real database (Docker container లో) test కోసం spin up చేసి, test తర్వాత automatic గా tear down చేస్తుంది — H2 యొక్క speed మరియు production DB యొక్క realism, రెండింటి మధ్య best balance.

Professional English Explanation

@DataJpaTest defaults to H2 (in-memory DB) — fast, but **not 100% behaviorally identical** to a production database (PostgreSQL, MySQL) — SQL dialect differences, constraint behaviors.

Testcontainers spins up a real database (in a Docker container) for testing, tearing it down automatically afterward — the best balance between H2's speed and a production database's realism.

Java Code

```
import org.testcontainers.containers.PostgreSQLContainer;
import org.testcontainers.junit.jupiter.*;
import org.springframework.test.context.DynamicPropertyRegistry;
import org.springframework.test.context.DynamicPropertySource;
import org.springframework.boot.test.context.SpringBootTest;
import org.junit.jupiter.api.Test;

@SpringBootTest
@Testcontainers // activates
Testcontainers JUnit 5 extension
class OrderRepositoryTestcontainersTest {

    @Container // Testcontainers
    manages this container's lifecycle
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16-alpine")
        .withDatabaseName("testdb")
        .withUsername("test")
        .withPassword("test");

    @DynamicPropertySource // wire the REAL
    container's connection into Spring
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl); // dynamically-
        assigned port, since Docker picks one
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @org.springframework.beans.factory.annotation.Autowired
    private OrderRepository orderRepository;

    @Test
    void savesAndRetrievesOrder_againstRealPostgreSQL() {
        Order order = new Order("ORD-1", 500.0);
        orderRepository.save(order);

        java.util.Optional<Order> found = orderRepository.findById(order.getId());
        org.junit.jupiter.api.Assertions.assertTrue(found.isPresent());
        // This test exercises REAL PostgreSQL-specific behavior (types, constraints,
        functions) -
        // something H2's PostgreSQL-COMPATIBILITY MODE can approximate but not perfectly
        guarantee
    }
}
```

Internal Working

- `@Container` + `static` field means the PostgreSQL Docker container is (by default) started **once and shared** across all test methods in the class — Testcontainers manages the full lifecycle (pulling the image if needed, starting the container, waiting until the database is actually ready to accept connections, and tearing it down when the JVM exits) entirely automatically, requiring no manual Docker commands or CI-pipeline database provisioning steps.
- `@DynamicPropertySource` exists specifically because the container's actual connection details (particularly its **port**, since Docker typically assigns a random available host port to avoid

conflicts) aren't known until the container has actually started — this is why the datasource URL can't simply be a static `application-test.properties` value (Book 12, Ch.8); it must be wired in dynamically, *after* the container starts but *before* Spring's `ApplicationContext` needs the datasource configuration.

- Testing against the **actual production database technology** (not just an "H2 in PostgreSQL-compatibility mode" approximation) catches a genuine, real category of bugs: subtle SQL dialect differences, specific constraint enforcement behavior, database-specific functions (Book 09, Ch.9's native queries), and exact type-mapping edge cases that an in-memory approximation can silently mask — Testcontainers exists specifically because "tests passed against H2 but failed in production against real PostgreSQL" is a real, documented, historically common integration-testing gap.

Real-World Example

Telugu: Real production teams `application.properties` లో production database PostgreSQL అయితే, integration tests కూడా Testcontainers తో real PostgreSQL మీద run చేస్తారు — H2 తో test చేస్తే pass అయినా, production PostgreSQL-specific SQL syntax/constraint behavior తేడాలు వల్ల production లో fail అయ్యే bugs catch చేయబడతాయి. English: Real production teams whose production database is PostgreSQL run their integration tests against real PostgreSQL via Testcontainers — catching bugs that would pass against H2 but fail in production due to PostgreSQL-specific SQL syntax or constraint behavior differences, a real and well-documented integration-testing gap Testcontainers directly closes.

Interview Answer

"Testcontainers spins up a real database in a Docker container for integration testing, automatically managed (started, waited-for-readiness, torn down) by the test framework — closing the gap between H2's speed/convenience and a production database's actual behavior.

`@DynamicPropertySource` wires the container's runtime-assigned connection details (like its randomly-assigned port) into Spring's configuration after the container starts. This catches real bugs from SQL dialect differences, constraint behavior, and database-specific functions that an in-memory approximation can silently mask — a well-documented, real integration-testing gap Testcontainers directly addresses."

Cross Questions

- Q: Why can't the Testcontainers database's connection URL be a static property in `application-test.properties`? → A: The container's actual port (and sometimes host) isn't known until it has actually started, since Docker typically assigns a random available port — the connection details must be wired in dynamically after startup via `@DynamicPropertySource`.
- Q: What real category of bug does testing against H2 alone risk missing? → A: SQL dialect differences, specific constraint enforcement, database-specific functions, and type-mapping edge cases that differ between H2's approximation and the actual production database technology.
- Q: What does `@Container` on a static field mean for the container's lifecycle? → A: By default, the container is started once and shared across all test methods in that test class, rather than

restarted per test — a deliberate speed optimization for a genuinely expensive resource.

Tricky Questions

- Q: Does using Testcontainers mean you no longer need @DataJpaTest-style tests against H2 at all? → A: Not necessarily — H2-based tests remain valuable for their speed in the common case; Testcontainers-based tests are typically reserved for cases where genuine database-specific behavior matters, or as a smaller number of higher-confidence integration tests, following the test pyramid's (Ch.1) speed-vs-realism trade-off rather than replacing fast tests entirely.
- Q: Does Testcontainers require Docker to be installed and running wherever tests execute, including CI? → A: Yes — this is a genuine prerequisite/operational consideration; most modern CI platforms support Docker-in-CI, but it's a real dependency that must be accounted for when adopting Testcontainers, unlike H2 which requires nothing beyond the JVM itself.

Coding Exercise

L1: Set up a Testcontainers-based integration test using a real PostgreSQL (or MySQL) container. **L2:** Write a test exercising a database-specific feature (a specific function or constraint) that would behave differently or be unavailable in H2. **L3:** Research your CI platform's Docker support and note any configuration needed to run Testcontainers-based tests there. **L4 (Interview):** Explain what problem Testcontainers solves that plain H2-based testing doesn't. **L5 (Senior):** Design a testing strategy deciding which tests should use H2 versus Testcontainers for a project targeting PostgreSQL in production. **L6 (Mastery):** Explain, from memory, why @DynamicPropertySource is necessary for wiring a Testcontainers database into Spring's configuration.

CHAPTER 7 — API Testing & Test Strategy

Telugu Explanation

ఈ chapter లో "ఏమి test చేయాలో, ఎక్కడ test చేయాలో" decide చేసే **test strategy** thinking చూద్దాం — ప్రతి line of code కి 100% coverage వెంబడించడం సరైన goal కాదు; **risk-based testing** (business-critical, complex, frequently-changing code కి ఎక్కువ test focus), test **fragility** avoid చేయడం (implementation details కి కాకుండా behavior కి test చేయడం).

Professional English Explanation

This chapter covers **test strategy** thinking — deciding what to test and where. Chasing 100% line coverage on every piece of code isn't the right goal; **risk-based testing** (focusing test effort on business-critical, complex, frequently-changing code) and avoiding test **fragility** (testing behavior, not implementation details) are the more mature approach.

Java Code — Testing Behavior, Not Implementation

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

class TestStrategyDemo {

    static class OrderService {
        private final OrderRepository repository;
        OrderService(OrderRepository repository) { this.repository = repository; }

        double calculateTotal(java.util.List<Double> itemPrices) {           // PUBLIC BEHAVIOR -
the contract callers rely on
            double sum = 0;
            for (double price : itemPrices) sum += price;                 // IMPLEMENTATION
DETAIL - HOW it sums, could change
            return sum;
        }
    }

    // FRAGILE TEST (BAD): tests implementation details (verifying a private helper's exact
    call pattern)
    // - if the sum algorithm is refactored (e.g., to use Book 07's Streams:
    itemPrices.stream().mapToDouble(...).sum()),
    //   this kind of over-specified test could break EVEN THOUGH THE BEHAVIOR IS STILL CORRECT

    // ROBUST TEST (GOOD): tests the PUBLIC BEHAVIOR/CONTRACT only - survives internal
    refactoring
    @Test
    void calculateTotal_sumsAllItemPrices() {
        OrderService service = new OrderService(mock(OrderRepository.class));
        double total = service.calculateTotal(java.util.List.of(10.0, 20.0, 30.0));
        assertEquals(60.0, total, 0.001);                                // tests WHAT
it returns, not HOW it computes it
    }

    // RISK-BASED TESTING EXAMPLE: a payment calculation deserves MUCH more test depth than a
    simple getter
    static class PaymentCalculator {
        double calculateFinalAmount(double base, double taxRate, double discountRate) {
            double afterDiscount = base * (1 - discountRate);
            return afterDiscount * (1 + taxRate);
        }
    }

    @Test void payment_normalCase() {
        assertEquals(108.0, new PaymentCalculator().calculateFinalAmount(100.0, 0.08, 0.0),
0.01);
    }
    @Test void payment_withDiscount() {
        assertEquals(97.2, new PaymentCalculator().calculateFinalAmount(100.0, 0.08, 0.10),
0.01);
    }
    @Test void payment_zeroAmount_edgeCase() {                                // EDGE
CASE - business-critical, worth extra care
        assertEquals(0.0, new PaymentCalculator().calculateFinalAmount(0.0, 0.08, 0.10), 0.01);
    }
    @Test void payment_fullDiscount_edgeCase() {                                // 100%
discount - boundary condition
```

```

        assertEquals(0.0, new PaymentCalculator().calculateFinalAmount(100.0, 0.08, 1.0),
0.01);
    }
    @Test void payment_negativeTaxRate_shouldBeRejectedOrHandled() { //
defensive/invalid-input case
        // A financial calculation getting a NEGATIVE tax rate is almost certainly a bug
upstream -
        // this test documents/enforces the expected (defensive) behavior explicitly
        assertThrows(IllegalArgumentException.class,
            () -> new PaymentCalculator() {
                double calculateFinalAmount(double base, double taxRate, double
discountRate) {
                    if (taxRate < 0) throw new IllegalArgumentException("Tax rate cannot be
negative");
                    return base;
                }
            }.calculateFinalAmount(100.0, -0.05, 0.0));
    }
}

```

Internal Working

- **Fragile tests** (over-specifying *how* something is implemented, rather than *what* it produces) create a real, ongoing maintenance cost: every legitimate internal refactoring (Book 05, Ch.13's example of switching a data structure, or adopting Book 07's Streams for cleaner code) risks breaking tests that were never actually testing the wrong *behavior* — just an incidental implementation detail — this is exactly why testing through the **public contract** (inputs → outputs, and observable side effects for void methods, Ch.3) rather than internal mechanics is the mature, durable testing approach.
- **Risk-based test depth** means not every method deserves equally exhaustive coverage — a trivial getter/setter genuinely needs little to no dedicated testing (its correctness is nearly self-evident and low-risk if wrong), while a financial calculation, an authorization check (Book 14), or a concurrency-sensitive operation (Book 08) genuinely warrants deep, edge-case-heavy testing (zero values, boundary conditions, invalid input, negative numbers) — proportional to the real-world cost of that specific code being wrong.
- Chasing **100% line coverage** as a goal in itself is a well-known, genuine anti-pattern: it's entirely possible to have 100% line coverage while still having zero tests that actually verify meaningful edge cases or business rules correctly (a test can execute every line without asserting anything meaningful about the results) — coverage percentage is a useful *signal* for finding genuinely untested code, never a sufficient *proof* of test quality on its own.

Real-World Example

Telugu: Real production teams payment calculation, authorization logic, tax computation వంటి business-critical code కి extensive edge-case testing పెడతారు (zero, negative, boundary values), కానీ simple DTO getters కి దాదాపు test రాయరు — ఇదే risk-based testing యొక్క practical application. English: Real production teams apply extensive edge-case testing (zero, negative, boundary values) to business-critical code like payment calculations, authorization logic, and tax computation, while writing little to no dedicated testing for simple DTO getters — exactly risk-based testing applied in practice.

Interview Answer

"Good test strategy avoids fragile tests that over-specify implementation details rather than testing the public behavioral contract — fragile tests break during legitimate refactoring even when behavior stays correct, a real maintenance cost. Risk-based testing focuses depth proportional to real-world risk — business-critical calculations, authorization logic, and concurrency-sensitive code deserve deep, edge-case-heavy coverage, while trivial code needs little. Chasing 100% line coverage as a goal in itself is a known anti-pattern — coverage percentage finds untested code but never proves test quality, since a test can execute every line without asserting anything meaningful."

Cross Questions

- Q: Why does testing implementation details instead of behavior create a maintenance cost? → A: Legitimate internal refactoring (changing an algorithm or data structure while preserving correct behavior) can break over-specified tests that were never actually validating the wrong thing — just an incidental detail of how the old implementation worked.
- Q: Why doesn't 100% line coverage guarantee good tests? → A: A test can execute every line of code without making any meaningful assertions about correctness — coverage measures what code ran during tests, not whether the tests actually verify anything meaningful.
- Q: What determines how much test depth a piece of code deserves under risk-based testing? → A: The real-world cost/risk of that code being wrong — business-critical calculations and security-sensitive logic warrant deep, edge-case-heavy testing; low-risk trivial code needs proportionally less.

Tricky Questions

- Q: Is it ever appropriate to verify a specific internal method call sequence (an implementation detail) in a test? → A: Occasionally, yes — for code whose entire contract genuinely IS a specific interaction sequence (e.g., verifying a caching layer actually checks the cache before hitting the database, Ch.3's behavior verification), testing that interaction is testing the real behavior, not an incidental detail — the line between "behavior" and "implementation detail" depends on what the code's actual contract/purpose is.
- Q: Can a codebase have high test coverage numbers but still ship frequent production bugs? → A: Yes, and this is a real, documented pattern — if tests are fragile, low-quality, or missing genuinely important edge cases despite executing most lines, high coverage numbers can create false confidence while real risk-based gaps remain unaddressed.

Coding Exercise

L1: Identify a fragile, implementation-detail-testing example (provided or from your own past code) and rewrite it to test behavior instead. **L2:** Apply risk-based testing to a small module, writing extensive edge-case tests for its most business-critical method and minimal tests for a trivial getter. **L3:** Research your project's current test coverage percentage and critically evaluate whether it reflects genuine test quality or just line execution. **L4 (Interview):** Explain why 100% line coverage is not a sufficient goal for test quality. **L5 (Senior):** Design a risk-based test strategy for a new feature, explicitly identifying which parts deserve deep edge-case coverage and which don't, with

justification. **L6 (Mastery):** Explain, from memory, the distinction between testing behavior and testing implementation details, with your own fresh example of each.

CHAPTER 8 — Mini Project: Full Test Suite

Goal

Build a complete, production-quality test suite for a REST API combining Books 12-14's work (Spring Boot, JPA, Security), applying every concept from this book.

Requirements

1. **Unit tests** (Ch.1-4) for every `@Service` class's business logic, using Mockito to mock repository/collaborator dependencies, following AAA structure and FIRST principles throughout.
2. **@WebMvcTest** (Ch.5) for every `@RestController`, covering success responses, validation failures (400), not-found (404), and unauthorized/forbidden (401/403, Book 14) scenarios.
3. **@DataJpaTest** (Ch.5) for every custom repository query method, verifying correct results against a real (H2 or Testcontainers) database.
4. **At least one Testcontainers-based integration test** (Ch.6) exercising a database-specific behavior against the real production database technology.
5. **Parameterized tests** (Ch.2) for at least one validation or calculation method with multiple meaningful edge cases (boundary values, invalid input).
6. **A risk-based test-depth justification** (Ch.7): a short written rationale (in comments or a README) explaining which parts of the codebase received deep testing and which received lighter coverage, and why.
7. **At least one full @SpringBootTest integration test** exercising a complete request-to-database-and-back flow, including authentication (Book 14).
8. Verify the full suite runs quickly enough for practical CI use — measure and report the total execution time, and the proportion of tests at each pyramid level (Ch.1).

Concepts Reinforced

Every chapter in this book — the test pyramid, JUnit 5, Mockito, effective test writing, Spring Boot testing slices, Testcontainers, and risk-based test strategy — applied together to build genuine confidence in a real, secured, persisted REST API, tying together the work from Books 09-14.

Stretch Goal

Set up a CI configuration (conceptually, or actually if you have the means — Book 24 covers this fully) running the fast unit/slice tests on every commit and the slower Testcontainers/full integration tests on a less frequent schedule, reflecting the test pyramid's speed/cost trade-off in real pipeline design.

FINAL REVISION NOTES

- **Test pyramid:** many fast unit tests (no Spring context, Book 11's constructor-injection payoff) → some slice tests (one layer + mocks) → few slow integration/E2E tests; shape driven by speed/realism trade-off for CI/CD.
 - **JUnit 5:** `@BeforeEach` / `@AfterEach` (per-test, fresh instance) vs `@BeforeAll` / `@AfterAll` (once, static); `assertAll` reports all failures together; `@ParameterizedTest` avoids near-duplicate test methods.
 - **Mockito:** `mock()` = no behavior until stubbed; `verify()` = behavior verification for void/side-effect methods; `ArgumentCaptor` inspects actual arguments; `spy()` = real object with selective overrides, used sparingly.
 - **Effective tests:** AAA structure (Arrange/Act/Assert) for debuggability; FIRST principles — Fast, Independent (no shared mutable state), Repeatable (no hidden time/randomness/network dependencies), Self-validating, Timely.
 - **Spring Boot slices:** `@WebMvcTest` (web layer, services mocked, via `MockMvc` — no real server), `@DataJpaTest` (repository layer, auto-rolled-back transactions), `@JsonTest` (serialization only) — narrow, fast contexts.
 - **Testcontainers:** real database in Docker, closing the H2-vs-production behavioral gap; `@DynamicPropertySource` wires runtime connection details after container startup.
 - **Test strategy:** test behavior/contracts, not implementation details (fragility risk); risk-based depth (business-critical code gets deep edge-case coverage, trivial code doesn't); 100% coverage is not a sufficient quality goal.
-



CHEAT SHEET

```
Test pyramid: unit(MOST,fast,no Spring,Book11 payoff) > slice(some,1 layer+mocks) >
integration/E2E(FEW,slow,realistic)
JUnit5: @BeforeEach/@AfterEach(per-test,fresh instance) | @BeforeAll/@AfterAll(ONCE,static)
    assertThrows(Class, lambda) | assertAll(...) = ALL run, ALL failures reported |
@ParameterizedTest+@CsvSource/@ValueSource
Mockito: mock()=nothing until when().thenReturn() stubbed | verify()=BEHAVIOR
verification(void/side-effect methods)
    ArgumentCaptor=inspect actual args passed | spy()=REAL object, selective override via
doReturn().when() - use sparingly
AAA: Arrange-Act-Assert | FIRST: Fast,Independent(no shared state!),Repeatable(no
time/random/network),Self-validating,Timely
@WebMvcTest: web layer only, @MockBean services, MockMvc(no real server) | @DataJpaTest: repo
only, AUTO-ROLLBACK per test
@JsonTest: Jackson serialization only | TestRestTemplate: REAL HTTP, real server (full
@SpringBootTest)
Testcontainers: REAL DB in Docker | @DynamicPropertySource(port unknown until container starts)
| closes H2-vs-prod gap
Test strategy: test BEHAVIOR/contract not implementation details(fragility) | risk-based
depth(critical code=deep, trivial=light)
    100% coverage != quality proof, just "code ran," not "asserted correctly"
```



INTERVIEW QUESTION BANK — Java Testing

Beginner

1. What is the test pyramid, and why does it have that shape?
2. What is the difference between `@BeforeEach` and `@BeforeAll`?
3. What is a mock, and what does an unstubbed mock method return?
4. What is the AAA pattern?
5. What is the difference between `@WebMvcTest` and `@SpringBootTest`?

Intermediate 6. Explain the FIRST principles and why test independence specifically matters. 7. Explain the difference between state verification and behavior verification, with an example needing each. 8. Why does `@DataJpaTest` not require manual test cleanup? 9. What problem does Testcontainers solve that H2-based testing doesn't? 10. Why is 100% line coverage not a sufficient measure of test quality?

Advanced 11. Explain why constructor injection (Book 11) is what makes fast, Spring-context-free unit testing possible. 12. Explain the difference between testing behavior and testing implementation details, and the maintenance cost of the latter. 13. Explain how `@DynamicPropertySource` works with Testcontainers and why it's necessary. 14. Explain risk-based testing and how you'd decide test depth for a new feature. 15. Diagnose a flaky test (passes/fails intermittently) — walk through the FIRST-principle-based investigation process.

Senior/Architect 16. Design the complete test strategy (pyramid distribution, what to test at each level) for a new microservice. 17. Review a codebase with 95% integration tests and 5% unit tests — propose a rebalancing plan and justify it. 18. Design a CI pipeline strategy balancing fast feedback (unit/slice tests on every commit) against thorough coverage (Testcontainers/E2E on a schedule). 19. Explain end-to-end how you'd build a full test suite for a new secured (Book 14), persisted (Book 13) REST API feature from scratch. 20. Review a test suite with high coverage numbers but frequent production bugs — diagnose the likely root causes using this book's concepts.

(Full short/professional/deep-senior answer scaffolding for every question across the series lives in Book 23 — Java Interview Master Book.)



CROSS-QUESTION ENGINE — Sample Chains

Q: What is the test pyramid? → Q: Why are unit tests fastest? → Q: What specifically enables fast unit testing in a Spring application? → Q: Why not just write everything as integration tests for maximum confidence? → Q: What's an inverted pyramid, and why is it a problem?

Q: What is Mockito, and how does verify() differ from assertEquals()? → Q: What does an unstubbed mock return? → Q: When do you need behavior verification instead of state verification? → Q: What is ArgumentCaptor for? → Q: How does spy() differ from mock()?

Q: Why use Testcontainers instead of just H2? → Q: What specific category of bug can H2 miss? → Q: How does @DynamicPropertySource work with it? → Q: Does this mean you should stop using H2 entirely? → Q: What's the trade-off of Testcontainers versus H2, per the test pyramid?



CONSOLIDATED EXERCISES (All Levels)

L1 — Beginner: Redo each chapter's L1 exercise unaided. **L2 — Intermediate:** Redo each chapter's L2/L3 exercises, explaining every testing mechanic out loud in Telugu. **L3 — Advanced:** Write a complete test suite (unit + slice + integration) for a small feature from scratch. **L4 — Interview:** Answer all 20 Interview Bank questions from memory, under 90 seconds each. **L5 — Senior:** Complete the Chapter 8 mini project fully, including the CI stretch goal. **L6 — Mastery:** Teach Chapters 1 (test pyramid), 3 (Mockito), and 7 (test strategy) out loud, from memory, using fresh examples.



ONE-DAY REVISION PLAN (≈5 hours)

Time	Focus
0:00–0:30	Ch.1: Test pyramid — redraw the diagram from memory
0:30–1:00	Ch.2: JUnit 5 — lifecycle, assertions, parameterized tests
1:00–1:30	Ch.3: Mockito — the highest-density interview block
1:30–1:45	Break
1:45–2:15	Ch.4: Effective unit tests — AAA and FIRST
2:15–2:45	Ch.5: Spring Boot testing slices
2:45–3:15	Ch.6: Testcontainers
3:15–3:45	Ch.7: Test strategy — behavior vs implementation, risk-based depth
3:45–4:30	Ch.8: Mini project walkthrough
4:30–5:00	Interview Bank — answer all questions from memory



ONE-WEEK MASTER REVISION PLAN

Day	Focus
1	Ch.1–2 (pyramid, JUnit 5) — write tests using every lifecycle annotation and assertion type
2	Ch.3 (Mockito) — build a full mock-based unit test suite for a service with side effects
3	Ch.4 (effective tests) — refactor poorly-structured tests using AAA and FIRST
4	Ch.5 (Spring Boot slices) — write @WebMvcTest, @DataJpaTest, @JsonTest for one feature
5	Ch.6 (Testcontainers) — set up and run a real-database integration test
6	Ch.7–8 (test strategy) + Mini Project — build the complete test suite
7	Full mock interview: all 20 bank questions + all cross-question chains, in Telugu and English, without notes



FINAL MASTERY CHECKLIST

- ☐ I can explain the test pyramid and why it has that shape.
- ☐ I can use JUnit 5's lifecycle annotations, assertions, and parameterized tests correctly.
- ☐ I can use Mockito for mocking, stubbing, and behavior verification.
- ☐ I can write tests following AAA structure and all 5 FIRST principles.
- ☐ I can use @WebMvcTest, @DataJpaTest, and @JsonTest appropriately.
- ☐ I can set up and use Testcontainers for real-database integration testing.
- ☐ I can distinguish testing behavior from testing implementation details.
- ☐ I can apply risk-based test strategy, justifying test depth by real-world risk.
- ☐ I built the Full Test Suite mini project, including the CI stretch goal.
- ☐ I answered the full Interview Question Bank confidently in both Telugu and English.

Once every box is checked, you are ready for `16_Microservices.md`.